Analysis of Drift in Transportation Data Streams for Regression Tasks

Klaudia Kwoka, Pola Mościcka

Abstract

This project explores machine learning methods for data streams. It focuses on the regression task of predicting travel time across different transport modes, such as taxis and airplanes. It investigates drift in streaming data, with focus on spatial and seasonal drift. This study compares multiple models to evaluate and compare their predictions.

1 Introduction

Predicting travel time is a complex task because conditions such as traffic jams or bad weather are constantly changing. In machine learning for data streams, these unexpected changes are known as concept drift. This project aims to build an adaptive online learning system that can detect these shifts and accurately predict travel durations for different modes of transport.

To test our approach we use a combination of real and synthetic datasets covering two major transportation domains: NY city taxis and USA airplanes. Because real-world drift can be difficult to measure, we also developed synthetic datasets to simulate specific, controlled changes. This involves simulating trips across Warsaw, where we artificially introduce rush hours and weather conditions to impact travel time.

A core part of our strategy is partitioning these datasets into specific geographical areas. This spatial division allows our system to catch local changes early and use specialized local models to handle the evolving data streams more effectively.

2 Related work

In the paper [3] the authors investigated the impact of concept drift on flight delay prediction. Furthermore, they analyzed how the scale of the training data, comparing models trained on single airports versus the entire aviation system, affects the overall effectiveness of the models. The authors found that the optimal algorithm choice depends heavily on this spatial scale, as simple models excelled at the macro-level, whereas ensemble methods like Random Forests were more effective for localized data subsets. They also highlighted the stability-plasticity dilemma, warning that overly sensitive drift detection and excessively frequent model retrains can cause the algorithm to forget learned patterns and ultimately degrade predictions. It is important to note that, unlike our approach which uses online learning, the methodology in the paper relied on periodic retraining of models using static data batches. The experiments were based on a classification task and were evaluated specifically on a Brazilian flights dataset.

In [2], the authors proposed a chained prediction approach using multi-label Random Forest classification to iteratively forecast arrival and departure delays along an aircraft's daily itinerary. Through an optimal feature selection process, they demonstrated that the most critical predictors of flight delay are the aircraft's prior delay status and the available turnaround time between landing and departure to absorb that delay. By linking consecutive flights, their approach effectively captures the mechanics of delay propagation and improves overall predictive accuracy. These findings directly informed our feature engineering strategy, a concept we adapt and significantly expand upon in Section 4.

In [5], the authors introduce a new streaming algorithm called Reactive Robust Soft Learning Vector Quantization (RRSLVQ). It improves upon the standard RSLVQ by using the Kolmogorov-Smirnov test to detect when data patterns change over time (concept drift). Instead of completely retraining the model when a change occurs, their method simply adjusts its "prototypes" (central

points representing different classes based on a Gaussian Mixture model) to follow the shifting data. By updating only specific parts of the model, it successfully balances learning new information with remembering the old ones. As a result, the model should stay highly accurate on past data while easily adapting to new trends as they appear.

In [4], the author introduces a Frequency Filtering Metadescriptor that detects concept drift by transforming feature vectors into the frequency domain using the Fast Fourier Transform on data chunks. While their batch-processing approach differs from our continuous online learning framework, both aim to effectively manage evolving data streams. Particularly relevant to our work is their introduction of centroid drift detection, a concept that aligns with our localized drift monitoring strategy (see Section 5.3).

3 Datasets

For evaluation, both real-world and synthetic datasets were used to assess model performance under different conditions. This combination allowed for testing in realistic scenarios as well as controlled environments where specific data characteristics and drift patterns could be simulated.

3.1 Real datasets

3.1.1 Taxi dataset (2016 NYC Yellow Cab trip record data [6])

The dataset contains taxi trip records in New York City from 2016 published by New York City Taxi and Limousine Commission [1]. It includes approximately 1.5 million examples. Each observation describes a single taxi ride and consists of eight features, including vendor identifier, pickup timestamp, passenger count, spatial coordinates of pickup and drop-off locations and additional attribute indicating whether the trip data was temporarily stored in the vehicle before being transmitted. The target variable is the trip duration in seconds.

3.1.2 Airline dataset (Data Expo 2009: Airline on time data [1])

This dataset contains large-scale flight and transportation records collected in the United States. Each record describes a single flight and includes variables such as origin and destination airports, scheduled and actual departure/arrival times, and other operational attributes. The dataset has around 2.4 million records. The target variable is the arrival delay, which measures the difference in minutes between the scheduled and actual arrival times of flights.

3.2 Synthetic datasets

3.2.1 Warsaw dataset: Simulated trips across Warsaw

This dataset was synthetically generated with `osmnx` Python library[2]. First, a set of 300 thousand timestamps was generated from the date range between 2025-01-01 to 2025-12-31, with defined weighted distribution over months, days and hours. Then, for each of the timestamps, start and end coordinates were sampled by selecting nodes from the road network graph for Warsaw. This ensures that trips are sampled from actual roads. For each trip the shortest path is calculated from the `osmnx` graph. The distance is scaled by random value sampled from the range [1.0, 1.15]. Travel time is calculated by dividing distance by randomly sampled speed between 20 and 50 km/h. Rush hour periods are defined for hours 7-9AM and 4-6PM during which travel time is increased by 50%. For weekend trips the time is reduced by 15%. Weather conditions (['clear', 'rain', 'snow', 'fog']) are created in the dataset with different impact on the travel time. Based on the timestamp the following features are created: ['hour', 'day_of_week', 'is_weekend', 'is_holiday']. The target is travel time in minutes.

[1] https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
[2] https://osmnx.readthedocs.io/en/stable/

3.2.2 Modification of Airline dataset

Synthetic version of the Airline dataset was created, by modifying the real version of the dataset described in Section 3.1.2. For a selection of dates '2008-02-15' to '2008-02-20' in region 'Northeast' all the flights destinations were changed to the 'MA' airport. For all these flights the previous departure delay feature (see Section 4) was increased by 30 minutes as well as the target variable. Same process was conducted for the period '2008-02-08' to '2008-02-10' in the district South - increasing the delay and redirecting all the flights within this district to only one airport located withing the region.

4 Data preprocessing

4.1 Dividing into regions

In all datasets, during the preprocessing phase of arriving observations, the data is partitioned into regions. For each dataset, the process checks whether a trip, based on its pickup and dropoff coordinates, belongs to a specific district and, if so, to which one. These features are used only to assign incoming observations to the correct local pipeline (see Section 5.2) and are not later used in the test-then-train approach. If a trip does not belong to exactly one region, it is assigned to the Global region.

For the Taxi dataset (see Section 3.1.1), the division is based on the NYC boroughs, and observations are mapped into the districts of Manhattan, Brooklyn, and Queens. Originally, there should also have been Bronx and Staten Island regions, but the number of trips in these districts was extremely small (approximately 50–200 trips), so these regions were excluded. This is because the dataset contains yellow taxi rides, which mostly cover Manhattan. The mapping was performed using the pickup and dropoff latitude and longitude coordinates together with the `geopandas` library. The polygon boundary coordinates were obtained from the archived data provided by the NYC Department of City Planning from 2008.[3].

For the Warsaw trip dataset (see Section 3.2.1), the division was based on whether the trip took place on the left or right side of the Wisła River. The mapping was performed using the pickup and dropoff latitude and longitude coordinates together with the `geopandas` library and Warsaw polygon boundary coordinates [4].

A similar approach was applied to the Airline dataset (see Section 3.1.2). The data was divided into four major U.S. regions: South, West, Midwest, and Northeast. The assignment was based on the states to which the departure and arrival airports belong. The regional division follows the classification provided by the United States Census Bureau[5]. The same logic was also applied to the Synthetic Airline dataset (see Section 3.2.2).

More detailed partitions were not considered practical, as they would cover less than 20% of all trips. For example, restricting the Airline dataset to flights within a single state would account for less than 15% of the observations. In the Taxi dataset, more than 80% of trips occur within a single district, while in the Airline dataset more than half of the flights belong to a single region under the chosen regional division. The same observation applies to the synthetic datasets. These results indicate that the selected partitioning strategy provides a good balance between regional specificity and sufficient data coverage within each local pipeline.

4.2 Feature engineering

For the Airline dataset additional two features were introduced. The crutial aspect of delay prediction is knowledge whether the plane's previous flight was delayed and how much (see [2]). Based on the scheduled and actual departure times of the previous flight, we created the feature `previous_dep_delay`. We used departure times because, in real-world scenarios, this information is usually available earlier than arrival times. This also allows the model to account for situations where the actual flight duration may differ from the scheduled one, for example due to weather conditions.

We also introduced another feature - `scheduled_turnaround`, which represents the time in minutes between the scheduled departure of the current plane's flight and the scheduled arrival of the previous

[3] https://www.nyc.gov/content/planning/pages/resources/datasets/borough-boundaries#archive
[4] https://github.com/andilabs/warszawa-dzielnice-geojson
[5] https://en.wikipedia.org/wiki/List_of_regions_of_the_United_States

plane's flight. This helps the model understand whether a delay from the previous flight is likely to affect the next one.

For almost all datasets, excluding the Warsaw dataset created for this study, only a selected subset of features was used for prediction. For Airline and Synthetic Airline dataset it was: CRSDepTime, CRSArrTime, CRSElapsedTime (three features related to scheduled flight time), Distance, origin_lat, origin_long, dest_lat, dest_long (coordinates of the airpoirts instead of names of the airpoirts), previous_dep_delay, scheduled_turnaround', is_weekend - feature created based on the day of the week. Regarding the Taxi dataset we selected features: pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude', is_weekend - feature created based on the day of the week.

5 Methodology

5.1 Setup

The experiments are implemented in Python using `river` library for online learning. Data generation and preprocessing are performed using `osmnx`, `geopandas`, `geopy`,`numpy`, and `pandas`. The source code is available in GitHub repository `https://github.com/klaudiakwoka/DataStreams`.

5.2 Local pipelines

As stated previously, instead of using a single model for all observations, the data was divided into geographic regions. During processing, each observation is labeled based on its origin and destination to determine whether the trip occurs within a single region and if so, which region it belongs to (see Section 4.1).

Each region has its own separate processing pipeline. Every local pipeline is associated with its own ensemble consisting of a model and drift detectors. Each Drift Adaptive Regression Ensemble assigned to a specific region learns only from the observations belonging to that region and predicts the target variable for this subset of data. Additionally, there is a separate model called "Global" for trips that do not occur entirely within a single region.

5.3 Drift detection

We considered several approaches for drift detection. Dividing datasets into regions and using local pipelines enables the detection of spatial drift. By comparing the behavior of local pipelines with the global pipeline, which handles trips between regions, it is possible to observe whether local drift also affects the global model. This may provide information about the severity and spread of the detected drift. Error-based drift handling is implemented within the Drift Adaptive Regression Ensemble (see Section 5.5). In all experiments, we used the ADWIN (Adaptive Windowing) method for concept drift detection[6]. ADWIN was selected because it is one of the most widely used and reliable methods for handling concept drift, while also providing an adaptive mechanism for updating the model based on prediction errors.

Another aspect of drift detection is virtual drift, which focuses on identifying changes in the distribution of incoming observations. Most virtual drift detectors (e.g., KSWIN) operate on a single feature and therefore analyze only marginal feature distributions. Since our datasets contain multiple features, selecting the most appropriate feature for drift detection is not straightforward. This becomes particularly difficult in cases where information is represented by several related features, such as pickup and dropoff coordinates described by latitude and longitude values. On the other hand, maintaining separate drift detectors for multiple features would significantly increase the computational cost.

Further approaches to detecting virtual drift are discussed in the next section.

5.4 Centroid Drift Detection

In this work we explore the Centroid Distance Drift Detector introduced in the literature [4]. This method detects changes in the input distribution $P(X)$ by tracking shifts in the feature space over time. However, it does not detect changes in the relationship between inputs and outputs. The main

[6] `https://riverml.xyz/dev/api/drift/ADWIN/`

idea is tracking the distance between the refrence vector of mean and the current within time-window vector of mean of incoming observations.

$$\mathbf{x}_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,d}) \in \mathbb{R}^d$$

$$X_{ref}^{(t)} = \{x_{t-w+1}, x_{t-w+2}, \ldots, x_t\}$$

$$X_{new}^{(t+1)} = \{x_{t+1}, x_{t+2}, \ldots, x_{t+w}\}$$

$$\boldsymbol{\mu}_{ref}^{(t)} = \left(\mu_{ref,1}^{(t)}, \mu_{ref,2}^{(t)}, \ldots, \mu_{ref,d}^{(t)}\right)$$

$$\boldsymbol{\mu}_{new}^{(t+1)} = \left(\mu_{new,1}^{(t+1)}, \mu_{new,2}^{(t+1)}, \ldots, \mu_{new,d}^{(t+1)}\right)$$

$$\mu_{ref,j}^{(t)} = \frac{1}{w} \sum_{i=t-w+1}^{t} x_{i,j} \quad \text{dla } j = 1, \ldots, d$$

$$\mu_{new,j}^{(t+1)} = \frac{1}{w} \sum_{i=t+1}^{t+w} x_{i,j} \quad \text{dla } j = 1, \ldots, d$$

$$D\left(\boldsymbol{\mu}_{ref}^{(t)}, \boldsymbol{\mu}_{new}^{(t+1)}\right) = \left\|\boldsymbol{\mu}_{ref}^{(t)} - \boldsymbol{\mu}_{new}^{(t+1)}\right\|_2$$

In our approach, the reference set ($X_{ref}^{(t)}$) remains fixed over time and is defined based on the warm-up period - the initial segment of incoming observations (e.g., the first 30 days of data).

Additionally, normalization is performed using the mean and standard deviation computed from this warm-up (reference) period, and these values are kept constant throughout the entire experiment.

The computed euclidean distance is used to update the Page–Hinkley detector [7], which in our algorithm is configured with parameters `ph_min_instances`, `ph_delta`, `ph_threshold`, and `ph_alpha` - corresponding to the Page-Hinkeley parameters. The detector is updated at the end of each day. After the last observation of a given day, a daily centroid is constructed and the corresponding distance is computed. This value is then used to update the detector.

In this setting, the `ph_min_instances` parameter defines the minimum number of days that must pass before the detector is allowed to signal drift.

Algorithm 1 Centroid Drift Detector

Input: Feature vector $\mathbf{x}$, timestamp t, instance count n
Output: `drift_detected` $\in \{\text{True}, \text{False}\}$, `drift_log`
for *each observation* $(\mathbf{x}, t, n)$ **do**
  if *days_from_start*$(t) < T_w$ **then**
    $\mathcal{W} \leftarrow \mathcal{W} \cup \{\mathbf{x}\}$; `// warmup period - ref centroid`
  else
    if *reference not yet built* **then**
      $\boldsymbol{\mu}_{\text{ref}} \leftarrow \text{mean}(\mathcal{W})$ $\boldsymbol{\sigma}_{\text{ref}} \leftarrow \text{std}(\mathcal{W}) + \varepsilon$
    if *day*$(t) \neq$ *current_day* **then**
      $\mathbf{c} \leftarrow \text{mean}(\mathcal{C})$; `// daily centroid`
      $\hat{\mathbf{c}} \leftarrow (\mathbf{c} - \boldsymbol{\mu}_{\text{ref}}) \oslash \boldsymbol{\sigma}_{\text{ref}}$; `// normalisation`
      $\delta \leftarrow \|\hat{\mathbf{c}}\|_2$; `// distance from reference`
      PageHinkley.update(δ)
      if *PageHinkley.drift_detected* **then**
        `drift_log`.append({date, n, δ, $\mathbf{c}$}) `drift_detected` $\leftarrow$ True
      $\mathcal{C} \leftarrow \emptyset$
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{\mathbf{x}\}$
return *drift_detected, drift_log*

[7] https://riverml.xyz/dev/api/drift/PageHinkley/

5.5 Drift Adaptive Regression Ensemble

A custom ensemble for regression was developed and implemented by extending the `base.Regressor` interface (see Algorithm 2). The ensemble consists of multiple instances of the same base regressor. It is instantiated with one model. Additional models are added when concept drift is detected. The maximum number of models within the ensemble is controlled by `max_ensemble_size` parameter.

A helper class was created. It wraps the regressor together with its performance metric (instance of `river.metric`). Each member stores both the prediction model and the copy of evaluation metric used to assess its performance. This allows to monitor the performance of ensemble members and compare them when deciding which model should be removed from the ensemble.

For each incoming observation, all active members of the ensemble are updated. Predictions are made based on one of the following strategies specified by parameter `prediction_strategy`. The default one computes arithmetic mean. The second one computes a weighted mean, where models with better performance receive larger weights based on the prediction error. For error metrics, where bigger metric means that the model performance is worse, the weight is assigned based on the inverse of the error, and then normalized.

The ensemble uses concept drift detection to adapt to changes in the data distribution. The absolute error of the ensemble predictions is averaged over a window of size specified by parameter `window_size`. It is then used as the input to the drift detector.

Two separate detectors can be used, a warning detector and a drift detector. When a warning is detected, a shadow model is created. The shadow model is an untrained copy of the base estimator and is trained in parallel with active ensemble members. Its predictions are not included in the ensemble output. If drift is later confirmed, the shadow model is inserted to the actual ensemble.

When the maximum capacity of the ensemble is reached, the model that performs the worst is removed. The decision is based on the evaluation metric kept with each ensemble member. Additional parameter `retain_initial_model` allows to always keep the initial model in the ensemble.

To avoid unstability at the beginning of training, drift detection is started after a warmup period specified by parameter `warmup` that defines how many first samples are skipped before the drift detection starts.

All the drift events and shadow model insertions to the ensemble are logged to later analyse the adaptation process.

Algorithm 2 Drift Adaptive Regression Ensemble

Input: Base regressor f, drift detector D, warning detector W, metric M, prediction strategy S, max size K, warmup T, window size w
Output: Ensemble prediction $\hat{y}$
Initialize ensemble $\mathcal{E} \leftarrow \{f_1\}$
for *each observation* (x_t, y_t) **do**
  collect predictions $\{\hat{y}_{i,t}\}$ from all $f_i \in \mathcal{E}$
  compute ensemble prediction $\hat{y}_t$ using strategy S
  foreach $f_i \in \mathcal{E}$ **do**
    update metric M_i using $(y_t, \hat{y}_{i,t})$ update model f_i with (x_t, y_t)
  if f_s *exists* **then**
    update shadow model f_s with (x_t, y_t)
  compute ensemble error $e_t \leftarrow |y_t - \hat{y}_t|$ append e_t to W_e
  if $t > T$ ***and*** $|W_e| = w$ **then**
    compute average error $\bar{e}$ over W_e
    update detectors $D(\bar{e})$ and $W(\bar{e})$
    if *warning detected by* W ***and*** *not exists* f_s **then**
      create shadow model $f_s \leftarrow copy(f)$
    if *drift detected by* D **then**
      if f_s *exists* **then**
        add f_s to ensemble $\mathcal{E}$ $f_s \leftarrow \emptyset$
      if $|\mathcal{E}| > K$ **then**
        remove worst-performing member according to metric M
      reset detectors D and W
return $\hat{y}_t$

6 Experiments

The experiments use the test-then-train approach: the model first makes a prediction on a new piece of data and then it learns from the correct answer. Model performance is assesed with Root Mean Squared Error (RMSE) that heavily penalizes large prediction errors and allows to observe where the model is exstensively mistaken.

6.1 Experiment 1: Comparison of model perfomance

Initially linear regression was considered as a baseline. However after preliminary exepriments it was discarded due to poor performance. The evaluation compares the proposed DriftAdaptiveEnsemble (see Section 5.5) with Hoeffding Tree Regressor (HTR), Adaptive Random Forest (ARF) and Streaming Random Patches (SRP). Additionally, all models are compared against a simple baseline that always predicts the mean of the target variable. Since the ensemble uses drift detection to update its structure, we also test ARF and SRP in two setups: using the default `river` settings (without drift detection) and with the ADWIN drift detector added. Table 1 describes model settings used in the experiment. The settings that were different depending on the dataset used are included in Table 2.

At first, the experiment on the Warsaw dataset was conducted without setting the `max_depth` parameter. However, for the remaining two datasets, recursion limit issues occurred, which required limiting the maximum depth of the trees. Due to computational limitations, the experiments were conducted on the full Warsaw dataset, while only subsets of 500,000 samples were used for both the Taxi and Airline datasets. All three datasets were preprocessed as described in Section 4, with the exception that the assigned districts were omitted.

Table 1: Regression models evaluated in Experiment 1. ADWIN settings and base learner max_depth is specified in Table 2.

Model	Description	Configuration
MEAN	StatisticRegressor	statistic = `stats.Mean`
HTR	HoeffdingTreeRegressor	Default parameters
ARF	ARFRegressor	n_models = 10, seed = 17, warning_detector = None, drift_detector = None
ARF_drift	ARFRegressor with drift detection	n_models = 10, seed = 17, warning_detector=`drift.ADWIN`, drift_detector=`drift.ADWIN`
SRP	SRPRegressor	n_models = 10, seed = 17, model = `tree.HoeffdingTreeRegressor` warning_detector = None, drift_detector = None
SRP_drift	SRPRegressor with drift detection	n_models = 10, seed = 17, model = `tree.HoeffdingTreeRegressor` warning_detector=`drift.ADWIN`, drift_detector=`drift.ADWIN`
Ensemble	Proposed Drift Adaptive Regression Ensemble	base_estimator = `tree.HoeffdingTreeRegressor` warning_detector=`drift.ADWIN`, drift_detector=`drift.ADWIN`, metric = `metrics.RMSE`, max_ensemble_size = 10, retain_initial_model = False

Table 2: Number of processed samples and configuration settings used for each dataset.

Dataset	Processed samples	Drift detector	Warning detector	Max depth of ensemble member
Warsaw	300,000	`ADWIN(0.05)`	`ADWIN(0.005)`	None
Taxi	500,000	`ADWIN(0.01)`	`ADWIN(0.001)`	30
Airline	500,000	`ADWIN(0.01)`	`ADWIN(0.001)`	30

6.2 Experiment 2: Drift detection across districts

The next experiment focuses on analyzing drift detection methods. It compares the Ensemble model performance (RMSE error) within the defined districts and drift detection methods. This experiment was conducted in the same way on two real datasets - Taxi and Airline and on synthetic Warsaw dataset. For all datasets all of the observations were processed. The used models, drift detectors and parameters are presented in the Table 3.

Table 3: Experiment 2 - configuration of parameters per dataset.

Taxi dataset			
Method	parameter	model/detector	configuration
Model		Drift Adaptive Regression Ensemble	metric = metrics.RMSE, max ensemble size = 4, retain initial model = True
Ensemble	base_estimator	SRP	n_models = 2, seed = 17
Ensemble	drift_detector	ADWIN	delta = 0.05
Ensemble	warning_detector	ADWIN	delta = 0.5
Virtual drift	Centroid detector	Page-Hinkley	warmup_days = 30, ph_threshold = 2, ph_min_instances = 1, ph_alpha = 0.95, ph_delta = 0.01
Airline dataset			
Method	parameter	model/detector	configuration
Model		Drift Adaptive Regression Ensemble	metric = metrics.RMSE, max ensemble size = 4, retain initial model = True
Ensemble	base_estimator	SRP	n_models = 2, seed = 17
Ensemble	drift_detector	ADWIN	delta = 0.00001 for "West", "Northeast", "Midwest", "Global" districts, delta = 0.0005 for South district
Ensemble	warning_detector	ADWIN	delta = 0.005
Virtual drift	Centroid detector	Page-Hinkley	warmup_days = 30, ph_threshold = 2, ph_min_instances = 1, ph_alpha = 0.95, ph_delta = 0.01
Warsaw dataset			
Method	parameter	model/detector	configuration
Model		Drift Adaptive Regression Ensemble	metric = metrics.RMSE, max ensemble size = 5, retain initial model = True
Ensemble	base_estimator	SRP	n_models = 3, seed = 17
Ensemble	drift_detector	ADWIN	delta = 0.05
Ensemble	warning_detector	ADWIN	delta = 0.5
Virtual drift	Centroid detector	Page-Hinkley	warmup_days = 30, ph_threshold = 5, ph_min_instances = 1, ph_alpha = 0.95, ph_delta = 0.01

6.3 Experiment 3: Feature drift

This experiment was conducted on a synthetic dataset Airline. The purpose of the experiment was to evaluate the centroid drift detection method (see Section 5.4) and verify whether the introduced changes in feature distribution within specific districts and time periods could be successfully detected by the virtual drift detector. For all regions, the ensembles were configured using the parameters presented in the Table 4.

7 Results

7.1 Experiment 1

Figure 1 shows a comparison of models described in Table 1. The models were trained and on the whole Warsaw dataset. The plot displays RMSE error for the period from 2025-12-01 to 2025-12-31, where model training started on data from 2025-01-01. The results shows that all evaluated models perform better than the baseline mean predictor. The performance of both ARF and ARF_drift is nearly identical on average, which suggests that the ADWIN drift detection settings may not be sufficiently

Table 4: Experiment 3 - configuration of parameters.

	parameter	model/detector	configuration
Ensemble	base_estimator	SRP	n_models = 4, seed = 17
Ensemble	drift_detector	ADWIN	delta = 0.005 for "Manhattan", delta = 0.001 for "Queens", "Global", delta = 0.01
Ensemble	warning_detector	ADWIN	delta = 0.5
Virtual drift	Centroid detector	Page-Hinkley	warmup_days = 30, ph_threshold = 2, ph_min_instances = 1, ph_alpha = 0.995, ph_delta = 0.01

sensitive to detect concept drift in this dataset. Both SRP and SRP_drift achieve higher error than ARF models. The lowest error is observed for the Ensemble and HTR models. This may indicate that the current drift detection configuration is not optimal for this dataset. The good performance of the HTR model suggests that the dataset contains repeating patterns that are relatively easy to learn. As a result, the model can achieve good predictions without the need for frequent updates or replacement of ensemble members. This may explain why drift detection and ensemble adaptation do not provide a significant improvement in this experiment.

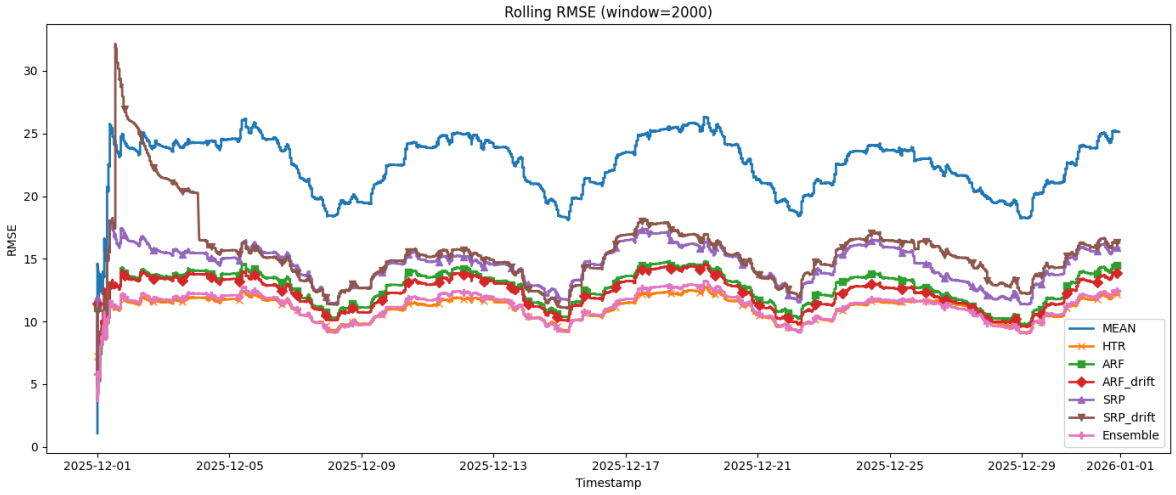

Figure 1: Comparison of model performance (see Section 6.1, Table 1) on the Warsaw dataset measured using RMSE over a window of 2000 observations for period from 2025-12-01 to 2025-12-31.

Figure 2 shows a comparison of the models described in Table 1 on the Airline dataset. All models were trained using the first 500,000 samples. The figure presents the RMSE calculated over a rolling window for the period from 2008-01-20 to 2008-01-25. The baseline mean predictor has the highest error. The ARF and ARF_drift models achieve very similar results, which again suggests that drift detection settings are not adjusted correctly to the dataset sample. The HTR and SRP_drift models perform better, achieving lower RMSE values. Interestingly, the SRP model without drift detection performs slightly better than SRP_drift. The Ensemble model achieves results similar to SRP.

7.2 Experiment 2

The plot presents RMSE results computed from the true and predicted values of the target variable over time, using sliding time windows that depend on the dataset and district (as specified in the `config_dataset_plot.json` file [8]). Additionally, the moments at which drift is detected are marked, distinguishing between the drift, warning, and centroid detectors.

[8] https://github.com/klaudiakwoka/DataStreams/blob/main/transformer/config_dataset_plot.json

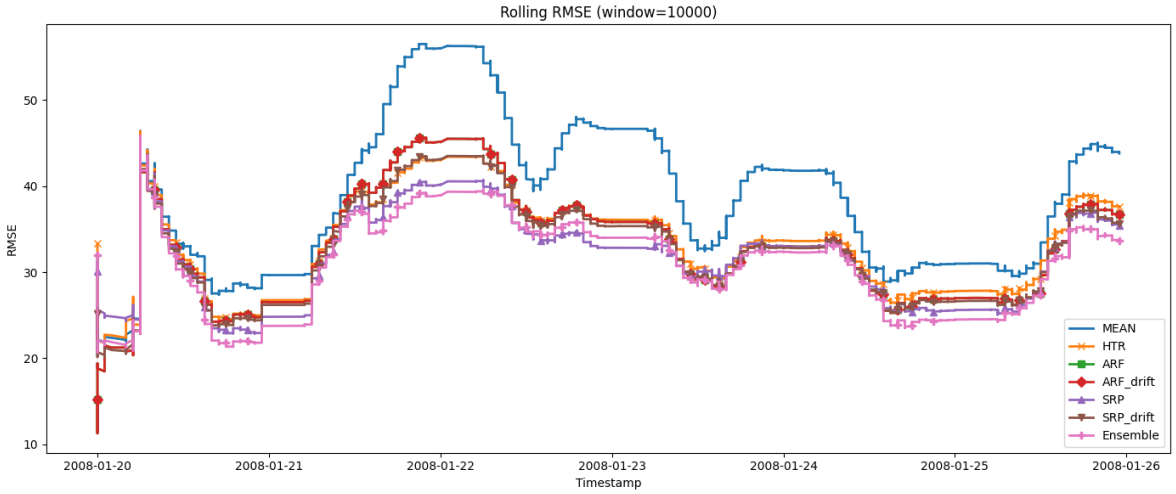

Figure 2: Comparison of model performance (see Section 6.1, Table 1) trained on a subset of Airline dataset (500,000 samples) measured using RMSE over a window of 10,000 observations for period 2008-01-20 to 2008-01-25.

Figure 3 illustrates the moments when drift was detected in the Warsaw dataset. The plot presents the November period after processing data collected since January. The samples represent all processed instances within the regions.

The drift was detected quite frequently, which indicates changes in the prediction errors made by the ensemble. The detected drift also depended on the region — drift observed in the Left district did not necessarily indicate changes in the Right district, which was the expected behavior. Drift was more common in the Global district, suggesting that dividing the dataset into regions may improve localization of drift detection and reduce false-positive detections. Overall, these results suggest that regional drift detection can better capture local changes in data behavior compared to a single global model.

The next figure 4 presents the results for the Taxi dataset for the last month of incoming data. Ensemble was trained from the beginning of the year, so by June the models are already well-adapted, which explains the relatively low baseline RMSE across most districts. ADWIN signals are frequent, particularly in Manhattan reflecting high error variability, while Centroid PH detects sparser feature drift — consistent with virtual drift occurring less frequently than prediction error fluctuations.

The results confirm that district-level models outperform the global one in terms of stability — local pipelines maintain consistently lower and smoother RMSE curves throughout the evaluated period. The Global district exhibits significantly more drift signals, likely including a high proportion of false positives, which reflects the difficulty of modelling cross-regional trips with a single model. Splitting the data by district reduces heterogeneity and leads to more reliable predictions and drift detection.

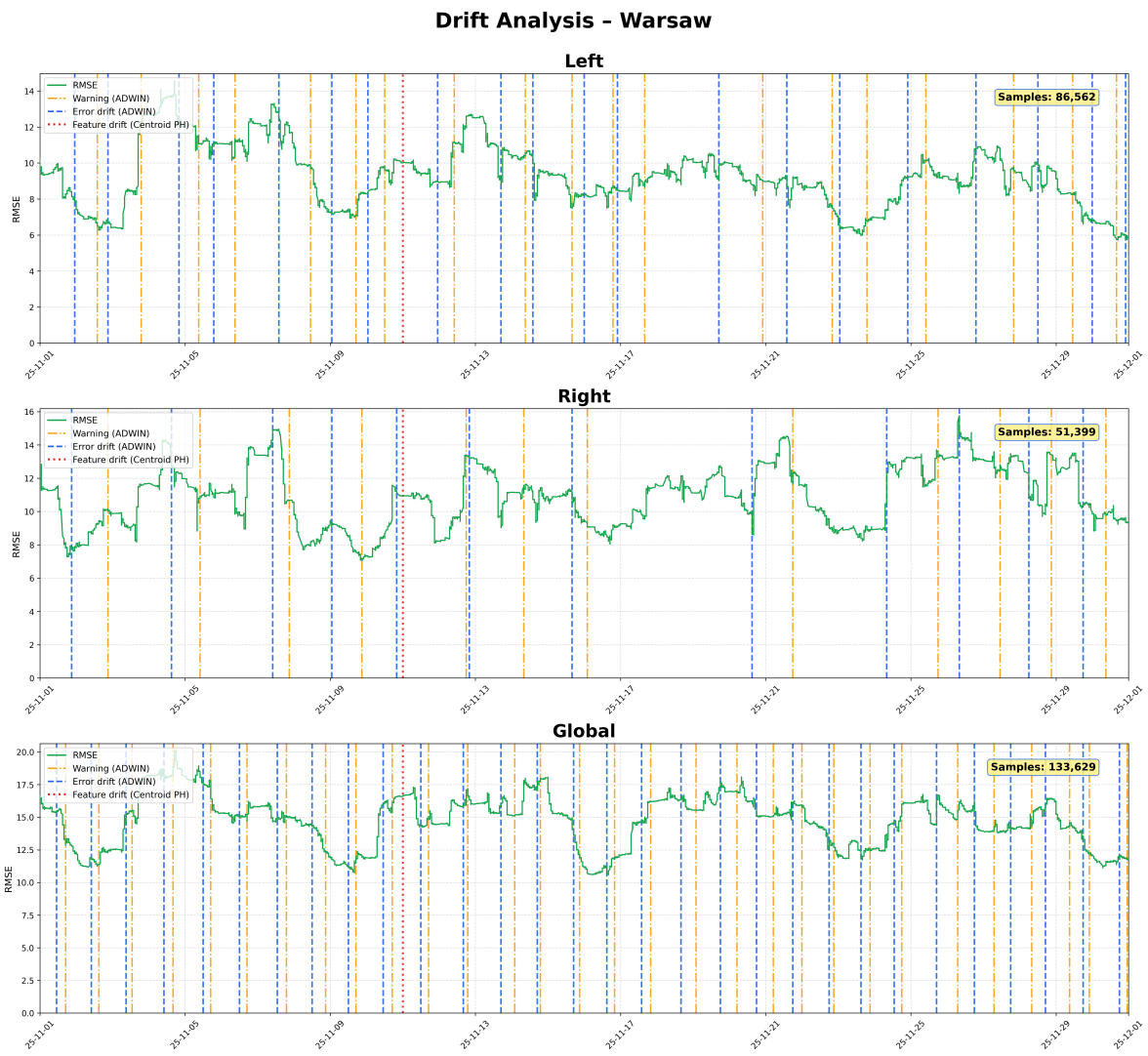

Figure 1: Drift detection and model performance analysis on Warsaw dataset.

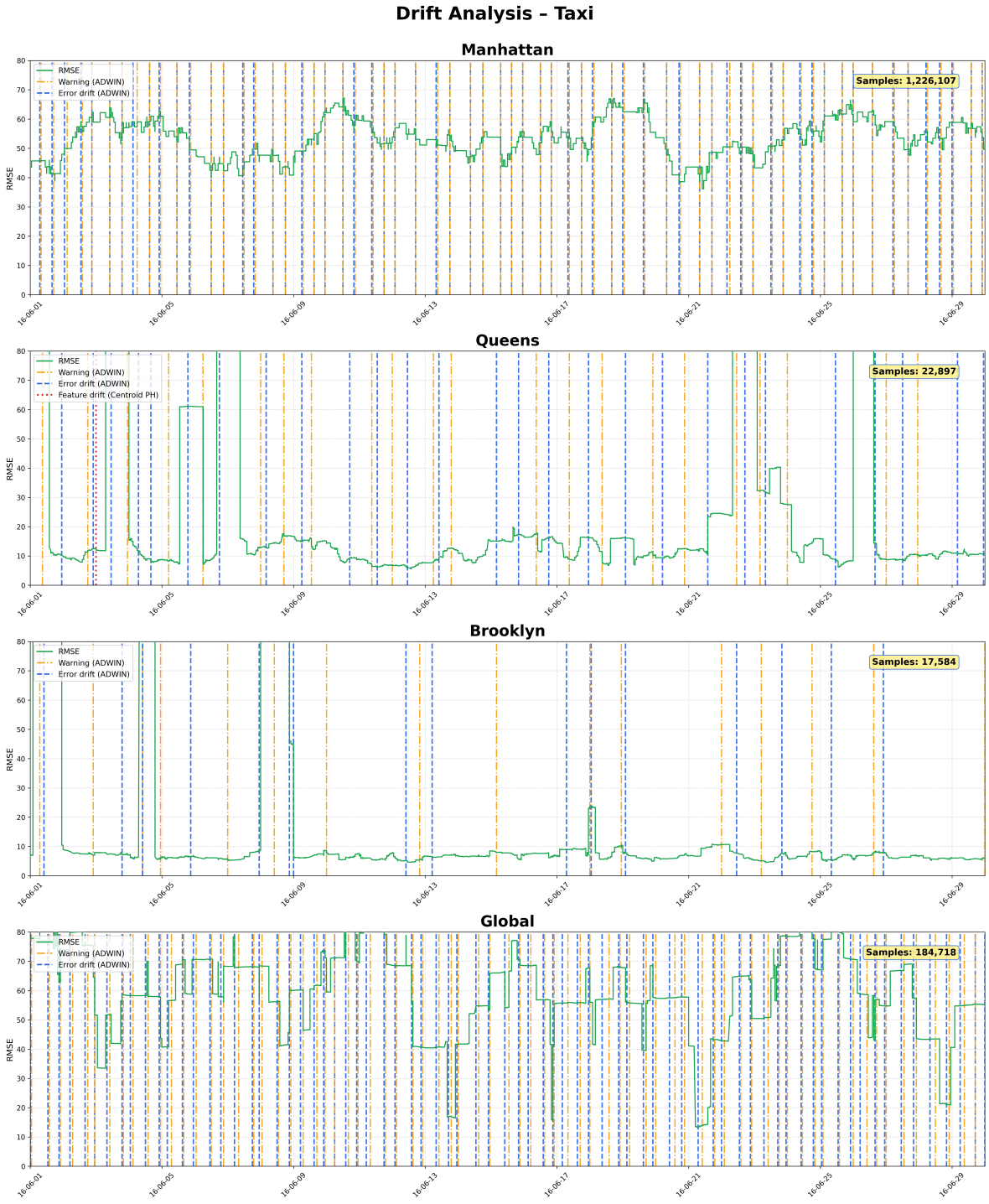

Figure 2: Drift detection and model performance analysis on Taxi dataset.

7.3 Experiment 3

Figure 5 presents RMSE results computed from the true and predicted values of the target variable over time, using sliding time windows that depend on the dataset and district (as specified in the `config_dataset_plot.json` file[11]). Additionally, the moments at which drift is detected are marked, distinguishing between the drift and centroid detectors. Periods when synthetic virtual drift occurs are marked in violet.

Figure shows that the centroid drift detector successfully identified the synthetic drift in the South and Northeast regions during the periods of intentionally introduced feature distribution changes (see Section 3.2.2).

The detected drifts appeared only in the affected districts and during or shortly after the expected time periods (8-9 February in the South region and 15–20 February in the Northeast region), while no significant drift was detected in the remaining regions.

8 Conclusions

Predicting data that changes every day is a difficult task because the patterns in the data keep shifting, which makes model performance unstable and can increase error.

In this report, a drift-adaptive regression ensemble model was developed and compared with several existing methods. The results showed that the proposed approach achieved competitive performance. Hovewer, proper drift detection configuration is important and simple models can perform very well on datasets with stable patterns. Future work would focus on testing the ensemble on the full datasets and exploring different drift detection settings to further improve its performance.

Additionally, virtual drift was investigated using the Centroid Drift Detector. In the real dataset, feature distribution shifts are infrequent compared to error-based signals and tend to be short-lived, with no consistent correlation with changes in model performance.

Furthermore, district-level model training and prediction yields more stable results than a single global model, reducing both prediction error and the frequency of spurious drift signals. The benefits become apparent over longer periods — although local pipelines learn from a smaller subset of data, they gradually converge to a better fit for trip duration and delay prediction within their respective regions.

9 Contribution of Authors

Table 5 summarizes the individual contributions to the project.

[11] https://github.com/klaudiakwoka/DataStreams/blob/main/transformer/config_dataset_plot.json

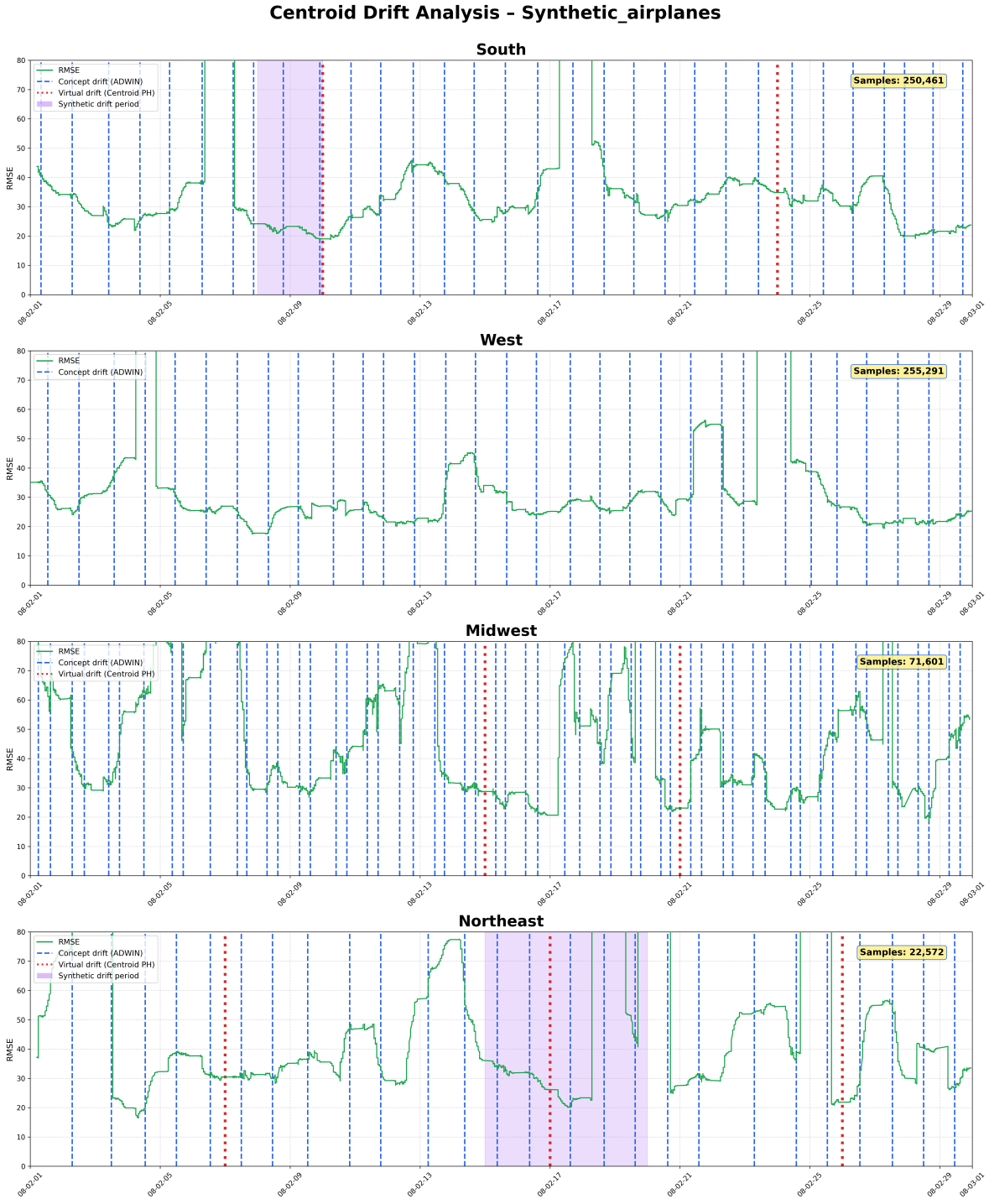

Figure 5: Centroid drift detection on Synthetic Airplanes across districts.

Task	Klaudia Kwoka	Pola Mościcka
Literature review	Research	Research and description in the report
Datasets	Generation of synthetic dataset of simulated Warsaw trip and its features. Modification of Airline dataset. Description in the report	—
Feature engineering	—	Preprocessing of the datasets. Splitting data into regions. Feature transformer for the datasets
Ensemble implementation	Design and implementation of the drift-adaptive ensemble, including the ensemble logic, prediction aggregation, and model update mechanisms	—
Drift detection	Integration of drift and warning detection within the ensemble	Implementation of centroid drift detection class. Experiments with drift detection
Experiments	Section 6.1	Section 6.2 and 6.3
Visualization	Joint contribution	Joint contribution
Analysis	Interpretation of results	Interpretation of results
Writing	Joint contribution	Joint contribution

Table 5: Contribution of each author to the project.

References

[1] Data Expo 2009: Airline on time data, 2008.

[2] Jun Chen and Meng Li. *Chained Predictions of Flight Delay Using Machine Learning*. 2019.

[3] Lucas Giusti, Leonardo Carvalho, Antonio Tadeu Gomes, Rafaelli Coutinho, Jorge Soares, and Eduardo Ogasawara. Analyzing flight delay prediction under concept drift. *Evolving Systems*, 13(5):723–736, January 2022.

[4] Joanna Komorniczak. Describing nonstationary data streams in frequency domain, 2025.

[5] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351, November 2020.

[6] Meg Risdal. New york city taxi trip duration. https://kaggle.com/competitions/nyc-taxi-trip-duration, 2017. Kaggle.